KubeAttackViz

Presentation & Live-Demonstration Plan

Kubernetes attack paths with ML-learned edge weights, validated against Kubernetes Goat and Bad Pods

Team (Group 39)	Roll no.
Ganesh Daware	23BCS038
Gagan Rathod	23BCS106
Venkatesh Dhangar	23BCS041
Advait Pathak	23BCS094

Course instructor: Dr. Malay Kumar · IIIT Dharwad

One command runs the whole demo: `./demo/present.sh`. One menu, five options, `s` then `a–d` switches cluster. Planned length about 15 minutes plus questions.

1. The story in one minute

The base paper (*Stay at the Helm*) builds a Kubernetes attack graph and ranks attack paths using **hand-set edge costs**. Those costs are assumptions: every privileged-looking edge costs roughly the same, whatever the evidence behind it. Our project keeps the paper's graph and replaces the costs with **learned success probabilities**:

	Before (paper-style, “old weights”)	After (our work, “ML weights”)
Edge weight	hand-set cost C	w = −ln P(success)
Path score	risk = impact / (1 + C/10)	risk = impact × P(path), P(path) = exp(−Σw)
Where P comes from	—	exact config (0.995) · CVE model · scenario parameter (0.05)
CVE model	—	gradient boosting on 361,825 NVD CVEs, CISA KEV labels; ROC-AUC 0.887 vs CVSS 0.770

Then we check the result against an answer key we did not write: the attacks that **Kubernetes Goat** and **Bad Pods** document for their own unmodified manifests. Finally we show it working in **real time**: a misconfiguration made in a running cluster is found and ranked within seconds.

Headline numbers to remember

Result	Old weights	ML weights
Documented answers matched (15 cases, full benchmark)	12/15	**14/15**
Mean rank of the 4 ranked Goat attacks (of 80 paths)	#17.5	**#2.5**
Bad Pods verdict correct	5/8	**7/8**
Real escapes ranked above non-escapes	62%	**91%**
Drift test (prod): new cluster-admin path	#17, MEDIUM	**#1, CRITICAL (P = 0.975)**
Time to detect the drift / the revert	16 s / 12 s	16 s / 12 s

2. Timeline and who speaks

Suggested split; swap names as you like. Each speaker drives the keyboard for their own part.

Time	Part	Screen	Speaker
0:00–1:30	Problem and base paper: assumed weights	slide / case study Fig. 1	Ganesh
1:30–3:00	What we changed: w = −ln P, CVE model	case study §4	Gagan
3:00–4:30	Step 1–2: the benchmark cluster, pods and RBAC	terminal: present.sh 1, 2	Venkatesh
4:30–8:00	Step 3: four dashboard views, old vs ML	terminal + browser: option 4	Gagan
8:00–10:00	Step 4: benchmark against Goat / Bad Pods	terminal: option 5	Advait
10:00–13:30	Step 5: real-time drift on prod	two terminals + browser	Venkatesh
13:30–15:00	Limitations, conclusion	paper Tables V–VI	Advait
15:00–	Questions (see §5)	—	all

3. Before the professor arrives (30 minutes earlier)

Do these on the laptop that will be presented. Everything is local (kind clusters in Docker); no internet is needed during the demo except to open the Goat / Bad Pods links.

```
cd ~/Desktop/7sem/cloud/Kuber_h2f
export PATH="$HOME/.local/bin:$PATH"  # kind lives here
docker ps                     # Docker must be running
kind get clusters             # expect kav-bench, kav-dev, kav-prod, kav-staging
./demo/setup.sh bench         # only if kav-bench is missing (1-2 min)
./demo/setup.sh prod          # only if kav-prod is missing
./demo/stop.sh                # stop any old dashboard server
python main.py run-tests      # optional: 31 tests should pass
```

Check	Why
Close other Chrome windows; zoom browser to 100%	the dashboard opens in a new window
Terminal font large (Ctrl + = a few times), window at least 120 columns	benchmark tables are wide
Open a second terminal tab in the same folder	used only for the drift command in Step 5
Revert any old drift: `./demo/drift.sh prod admin-sa revert`	prod must start clean, or the drift has nothing to show
Have docs/Case_Study_Grp39_Final.pdf and the IEEE paper open	backup if anything fails

4. Run sheet: exactly what to press

Start once; everything below happens inside this one menu.

```
./demo/present.sh
```

The menu shows the cluster table on top (▶ marks the current one) and five actions. Clusters: **a** = kav-bench, **b** = kav-dev, **c** = kav-staging, **d** = kav-prod. The menu starts on **a (bench)**.

Step 1 · Show the benchmark cluster: press 1 (about 1 min)

Lists the pods. They come from Kubernetes Goat (system-monitor, hunger-check, internal-proxy) and Bad Pods (priv, hostpid), applied **unmodified** from the upstream repositories.

> **Say:** *This cluster is not ours. These are public, intentionally vulnerable workloads whose authors already documented the attacks. That gives us an answer key we cannot tune.*

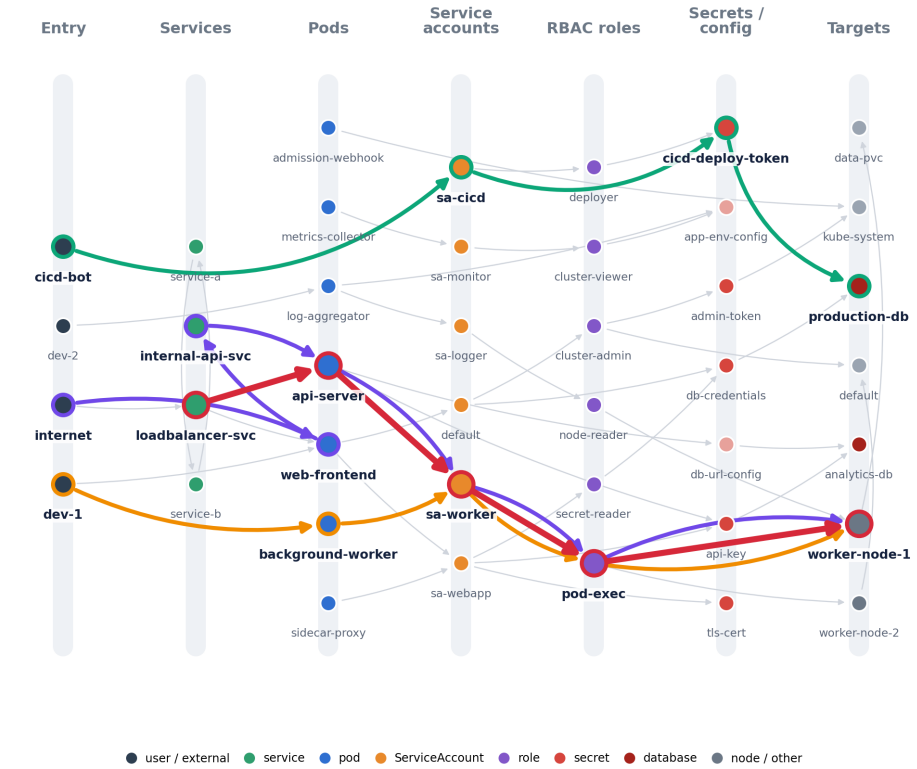
Step 2 · Who can do what: press 2 (about 1 min)

RoleBindings per namespace and the NetworkPolicies (none here, so every pod can reach every pod). Point at one binding that later appears in an attack path.

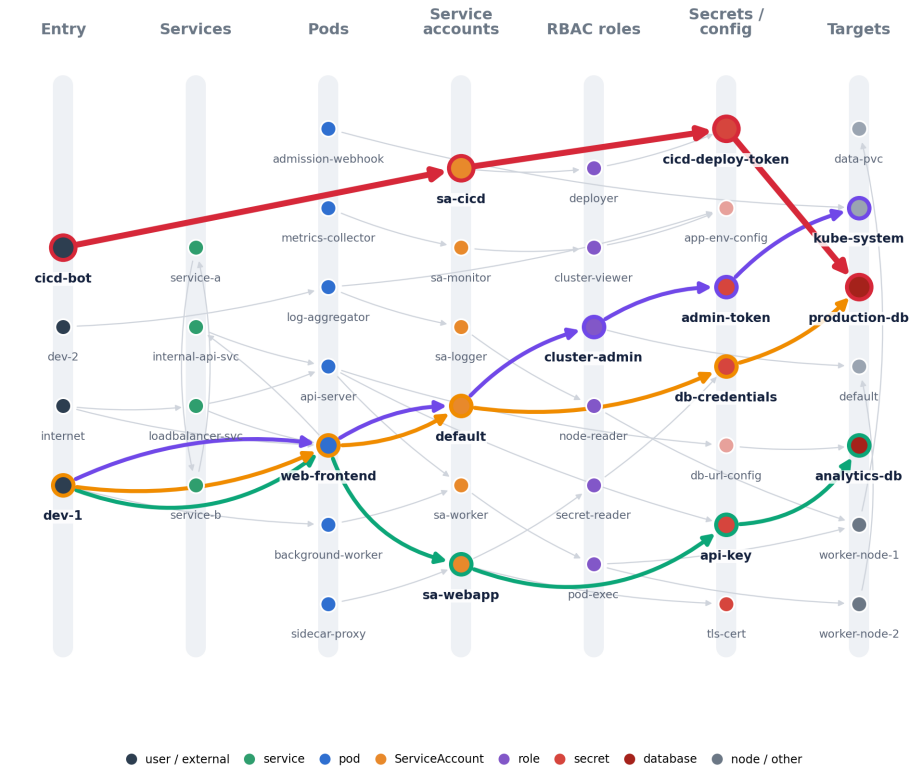
Step 3 · Old vs ML weights, four views: press 4 (about 3.5 min)

Starts the dashboard and opens Chrome. A small second menu appears in the terminal; each key changes the browser within a second. Old and ML views of the same cluster keep every node in the same place, so only the ranking changes.

Key	View	What to point at
1	sample cluster · old weights	the top-4 paths (red, orange, purple, green) in the caption
2	sample cluster · ML weights	the ▲ “was #k with old weights” tags: the order changes
3	live kav-bench · old weights	documented Goat paths sit low in the list
4	live kav-bench · ML weights	the same paths move to the top; Bad Pods priv reaches the node
q	back to the main menu	server keeps running



View 1: sample cluster with old weights.



View 2: same layout, ML weights; the ranked paths and their tags change.

> **Say:** *Same graph, same nodes, same edges. Only the weights changed, from assumed costs to learned probabilities, and the paths an attacker would really take rise to the top.*

If someone asks why some edges are missing: the drawing hides a few edges that are on no attack path, to keep the picture readable. The **All edges** button on the left rail shows every edge; the analysis always uses all of them.

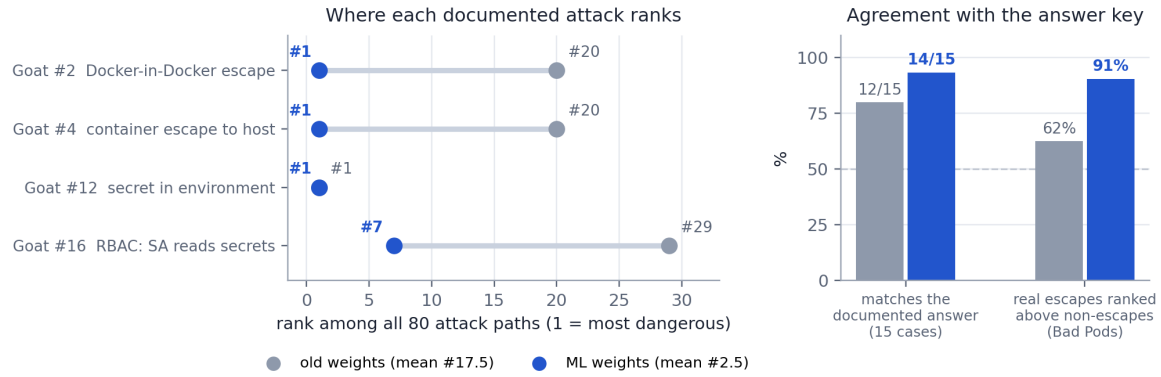
Step 4 · Benchmark: press 5 (about 2 min)

Runs the benchmark on the live kav-bench cluster and prints a Goat table (found, old rank, ML rank, ML probability), a Bad Pods table, a summary and a **Sources** section with ✔/✘ per item and the link to the page where Goat or Bad Pods documents that attack. Click one link to show it is real.

Then it prints a **See it live** link; press `Enter`. The browser opens on kav-bench (view 4, ML weights) with the **Benchmark: Goat + Bad Pods** tab: the same table, each row with its *source* link. Click a row and that documented path lights up in the graph. Press `3` / `4` in the terminal to compare old and ML weights on the same table (5/6 vs 6/6 match; Bad Pod #5 flips to correct).

Live demo set (12 nodes, 24 paths)	Old weights	ML weights
Mean rank of the deployed Goat attacks	#5.3	**#2.0**
Bad Pods verdict correct (priv, hostpid)	1/2	**2/2**

Then say the full-benchmark result (all of Goat + all 8 Bad Pods, run once with `BENCH_FULL=1`; it takes longer, so it is not run live):



Full benchmark: documented attacks rank #2.5 on average with ML weights, #17.5 with old weights; 14/15 vs 12/15 matches.

> **Say:** *Both weightings find all seven Goat attacks, so the graph was already right. The weights decide the order, and with ML weights the documented attacks are where a defender looks first.*

Step 5 · Real time: a misconfiguration appears in prod (about 3.5 min)

In the main menu press `s` then `d` (kav-prod, the hardened cluster), then `4`. Show view `4` (live prod, ML weights). In the **second terminal**:

```
./demo/drift.sh prod admin-sa        # bind shop/frontend SA to cluster-admin
```

Wait about 15 seconds. The live views rescan; an alert toast appears and a new path from the frontend to cluster-admin takes **#1, CRITICAL (P = 0.975)** in view 4. Press `3`: with old weights the same path is only **#17, MEDIUM**. Then undo it:

```
./demo/drift.sh prod admin-sa revert  # undo; seen in ~12 s, paths 21 -> 19
```

> **Say:** *Nothing was exploited; we only changed configuration, as a careless admin might. The tool noticed in 16 seconds, and only the learned weights put it at the top where it belongs.*

Close: press `q` in the views menu, then `q` in the main menu.

5. Questions to expect

Question	Short answer
Why w = −ln P?	Multiplying probabilities along a path becomes adding weights, so ordinary shortest-path search (Dijkstra) finds the most likely path. P(path) = exp(−Σw).
What does the ML model learn?	Whether a CVE gets exploited in the wild. Trained on 361,825 NVD CVEs, labelled by the CISA Known Exploited Vulnerabilities list (1,743 positives), split by time so it is tested on later CVEs. ROC-AUC 0.887 vs 0.770 for CVSS; in its top 100, 54% are exploited vs 5% for CVSS. Logistic regression reaches 0.877.
Is every edge from the model?	No. Edges fixed by configuration (a privileged pod reaches its node, an RBAC rule allows a read) are near-certain, 0.995. CVE edges use the model. Edges that depend on unknown conditions use a stated scenario parameter, 0.05.
Did you write the answer key?	We wrote it down, but its content is what Goat and Bad Pods document; every item links to their page (all 17 links checked). The manifests are unmodified, pinned to a commit.
Where does ML get it wrong?	Bad Pods hostPath: mounting the host filesystem can lead to the node, but only under conditions we cannot see in the spec, so it gets 0.05 and is not called an escape. Treating it as 0.05 is a deliberate, stated choice, not a tuned one.
Is the difference between old and ML too large?	The two rankings still agree on a lot: Spearman ρ = 0.71 / 0.84 / 0.57 on dev / staging / prod. They differ where the evidence differs, which is the point. The top path does not change when the scenario parameter goes up to 0.5.
Is this safe to run?	Everything is in local kind clusters. The drift script refuses any context that is not kind-kav-*. Delete the benchmark cluster after the demo.
Why only 12 nodes live?	So the graph is readable on a projector. The full benchmark (all Goat + all Bad Pods, 80 paths) gives the numbers in the paper.

6. If something goes wrong

Problem	Fix
`kind: command not found`	`export PATH="$HOME/.local/bin:$PATH"`; do **not** let dnf install kind (it wants to remove docker-ce)
A cluster shows stopped	choose it and press 1; present.sh starts it (about 30 s)
Browser does not open	open http://localhost:8765 by hand
Views 3–4 say “scanning…”	the first live scan takes 15–40 s with image scanning; talk over it
Dashboard shows the wrong cluster	`./demo/stop.sh`, then option 4 again
Drift toast does not appear	wait one more scan (15 s); check the second terminal printed no error
Nothing works	show the case study §7–8 and the paper Tables V–VI; screenshots are in there

7. After the demo

```
./demo/drift.sh prod admin-sa revert     # if not reverted yet
./demo/stop.sh                           # stop the dashboard server
kind delete cluster --name kav-bench     # it runs deliberately vulnerable workloads
```